

1 API Overview

This document describes the HTTP API for an online shop backend. The backend exposes endpoints for user accounts, authentication, hierarchical product categories, category-defined product attributes, products, carts, orders, mock payments, and product reviews.

The API is designed around common shop views and actions. Product listing views can request products directly, category views can request category trees and category-specific products, and product creation forms can request inherited category attributes before submitting product data. Most frontend views should be able to use one or two API calls without needing to understand the internal database structure.

1.1 Main Features

The backend currently supports the following functional areas:

- user registration, login, and authenticated profile access,
- administrator-managed users,
- nested product categories,
- product attributes defined by category inheritance,
- product creation, listing, filtering, updating, and archiving,
- shopping cart management,
- checkout with order creation,
- mock payment success and failure flow,
- reviews limited to users who bought and paid for a product,
- MongoDB validation, indexes, and transaction-based operations where needed.

1.2 Backend Responsibilities

The backend is responsible for enforcing the main business rules. The frontend may validate inputs for user experience, but the backend remains the source of truth.

Important backend-side rules include:

- only administrators can create, update, or archive products and categories,
- only administrators can create another administrator account,
- products must belong to an active category,
- product attribute values must match the attributes inherited from the selected category,
- checkout decreases product stock and creates both an order and a pending payment,
- users can review only products they bought and paid for,
- each user can have only one active review per product,
- archived and soft-deleted records are hidden from normal public endpoints.

1.3 Category and Product Attribute Model

Categories are hierarchical. A category may have a parent category, and every category stores its ancestor identifiers and depth level.

For example:

RTV

-> Computers

-> Keyboards

-> Smart Toasters

Categories also define product attributes. A child category inherits attributes from all of its ancestors.

For example, a root category may define general attributes:

- brand,
- warranty_months.

A child category may define more specific attributes:

- layout,
- switch_type,
- slices,
- burn_pattern.

A product does not define arbitrary attributes by itself. Instead, it stores values for attributes defined by its category chain.

```
{
  "categoryId": "66500000000000000000000001",
  "attributeValues": {
    "brand": "Samsung",
    "warranty_months": 24,
    "burn_pattern": "qr_code",
    "slices": 4
  }
}
```

This allows the frontend to build product forms dynamically. When an administrator selects a category, the frontend can request inherited attribute definitions from:

```
GET /categories/:id/attributes
```

The response tells the frontend which fields to render, which fields are required, which values are allowed, and which numeric constraints apply.

1.4 Typical Frontend Flows

1.4.1 Administrator creates a product

1. Fetch category tree using **GET** /categories/tree.

2. Administrator selects a category.
3. Fetch inherited attributes using `GET /categories/:id/attributes`.
4. Render product form fields based on returned attributes.
5. Submit product using `POST /products`.

1.4.2 User buys a product

1. Browse products using `GET /products`.
2. Add product to cart using `POST /cart/items`.
3. Review cart using `GET /cart`.
4. Checkout using `POST /orders/checkout`.
5. Complete mock payment using `POST /payments/:id/mock-success`.

1.4.3 User reviews a product

1. User completes checkout and payment.
2. User submits a review using `POST /products/:productId/reviews`.
3. Product rating summary is updated by the backend.

2 Common API Conventions

2.1 Base URL

During local development, the backend is available at:

`http://localhost:3000`

All endpoint paths in this document are written relative to this base URL.

For example:

`GET /products`

means:

`http://localhost:3000/products`

2.2 Content Type

All request bodies should be sent as JSON.

`Content-Type: application/json`

Unless stated otherwise, endpoints that accept a request body expect JSON.

2.3 Authentication

Authenticated endpoints require a Bearer token:

`Authorization: Bearer <accessToken>`

The API uses three practical access levels:

- **Public** — no token required,
- **User** — authenticated regular user,
- **Admin** — authenticated administrator.

Some endpoints are available to both users and administrators, but apply ownership rules. For example, a regular user can access only their own cart, orders, payments, and reviews, while an administrator may have broader access for selected operations.

2.4 Identifiers

All persistent entity identifiers are MongoDB ObjectId strings.

Example:

66500000000000000000000000000001

Path parameters named `id`, `productId`, `categoryId`, `orderId`, `paymentId`, or `reviewId` should be valid ObjectId strings.

Invalid ObjectId values are rejected before reaching the database layer.

2.5 Pagination

Paginated endpoints use the following query parameters:

- `page` — optional positive integer, default: 1,
- `limit` — optional positive integer, default: 20, maximum: 100.

Example:

`GET /products?page=1&limit=20`

Paginated responses use the following general shape:

```
{
  "items": [],
  "page": 1,
  "limit": 20,
  "total": 0,
  "totalPages": 0
}
```

2.6 Money Representation

Prices and monetary amounts are represented as integer-like numeric values in minor currency units.

For example:

```
{  
  "amount": 29999,  
  "currency": "PLN"  
}
```

means:

299.99 PLN

The currently supported currencies are:

- PLN,
- EUR,
- USD.

2.7 Common Status and Type Values

2.7.1 User roles

- user,
- admin.

2.7.2 User statuses

- active,
- deleted.

2.7.3 Category statuses

- active,
- archived.

2.7.4 Product statuses

- draft,
- active,
- archived.

2.7.5 Category attribute types

- string,
- number,
- boolean,
- string_array.

These types are used by category attribute definitions and determine how the frontend should render product attribute input fields.

Attribute type	Suggested frontend input
string with allowedValues	select/dropdown
string without allowedValues	text input
number	number input
boolean	checkbox/toggle
string_array with allowedValues	multi-select
string_array without allowedValues	tag input

Table 1: Frontend interpretation of category attribute types

2.7.6 Order statuses

- pending_payment,
- paid,
- payment_failed,
- cancelled,
- shipped,
- completed.

2.7.7 Payment statuses

- pending,
- paid,
- failed.

2.7.8 Review statuses

- active,
- deleted.

2.8 Common Error Response

Validation and business rule errors usually return a JSON response with a message, HTTP error name, and status code.

Example:

```
{
  "message": "Product not found",
  "error": "Not Found",
  "statusCode": 404
}
```

For DTO validation errors, the `message` field may be an array:

```
{
  "message": [
    "email must be an email",
    "password must be longer than or equal to 6 characters"
  ],
  "error": "Bad Request",
  "statusCode": 400
}
```

2.9 Database Validation Errors

Some write operations may fail MongoDB schema validation. In that case, the API returns a response with additional `validationErrors` details.

Example:

```
{
  "message": "Category failed database validation",
  "validationErrors": [
    "attributeDefinitions failed validation"
  ],
  "error": "Bad Request",
  "statusCode": 400
}
```

These errors usually indicate a mismatch between the service layer and the MongoDB collection validator. They are useful during development because they show which database-level rule rejected the document.

2.10 Soft Deletion and Archiving

The API generally avoids hard deletion for business entities.

- Users are soft-deleted by changing their status to `deleted`.
- Reviews are soft-deleted by changing their status to `deleted`.
- Products and categories are archived by changing their status to `archived`.

Public endpoints usually return only active records. Admin endpoints may expose inactive or archived records where needed.

2.11 Timestamps

Date-time fields are returned as ISO strings.

Common timestamp fields include:

- `createdAt`,
- `updatedAt`,

- `deletedAt`,
- `archivedAt`,
- `paidAt`,
- `failedAt`,
- `cancelledAt`.

Optional timestamp fields are returned only when relevant.

2.12 Endpoint Documentation Format

Each endpoint is described using the following structure:

- **Access** — public, authenticated user, administrator, or mixed ownership-based access.
- **Description** — what the endpoint does.
- **Query parameters** — optional URL parameters, if supported.
- **Path parameters** — required route parameters.
- **Request body** — JSON body accepted by POST and PATCH endpoints.
- **Important rules** — business rules or validation rules worth knowing.

3 Health Endpoints

Health endpoints are public utility endpoints used to check whether the backend application and database connection are available.

These endpoints do not require authentication.

3.1 GET /health

Access: Public

Description: Checks whether the backend application is running.

Request body: None.

Example request:

```
GET /health
```

Example success response:

```
{
  "status": "ok",
  "timestamp": "2026-05-30T10:00:00.000Z"
}
```

Response fields:

- **status** — application status. Returns `ok` when the backend is running.
- **timestamp** — ISO timestamp generated when the health check response is created.

3.2 GET /health/db

Access: Public

Description: Checks whether the backend can reach the configured MongoDB database.

Request body: None.

Example request:

```
GET /health/db
```

Example success response when connected:

```
{
  "database": "connected",
  "timestamp": "2026-05-30T10:00:00.000Z"
}
```

Example response when disconnected:

```
{
  "database": "disconnected",
  "timestamp": "2026-05-30T10:00:00.000Z"
}
```

Response fields:

- **database** — database connection status. Possible values: `connected`, `disconnected`.
- **timestamp** — ISO timestamp generated when the database health check response is created.

Important rules:

- This endpoint checks the database connection using a database ping.
- The endpoint is public so it can be used by simple monitoring tools or manual development checks.

4 Authentication Endpoints

Authentication endpoints are responsible for registering users, logging users in, and returning information about the currently authenticated user.

Successful registration and login return a Bearer access token. This token should be sent in the `Authorization` header when calling authenticated endpoints.

```
Authorization: Bearer <accessToken>
```

4.1 POST /auth/register

Access: Public

Description: Creates a new regular user account and immediately returns an access token.

Request body:

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Body fields:

- `email` — required string, must be a valid email address, maximum length: 254.
- `password` — required string, minimum length: 6, maximum length: 128.

Important rules:

- Registered users are created with the `user` role.
- Email addresses are normalized to lowercase.
- Email addresses must be unique.
- Public registration cannot create an administrator account.

Example request:

```
POST /auth/register
Content-Type: application/json
```

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Example success response:

```
{
  "accessToken": "<jwt>",
  "tokenType": "Bearer",
  "user": {
    "id": "665000000000000000000001",
    "email": "user@example.com",
    "role": "user"
  }
}
```

4.2 POST /auth/login

Access: Public

Description: Authenticates an existing active user and returns an access token.

Request body:

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Body fields:

- email — required string, must be a valid email address, maximum length: 254.
- password — required string, minimum length: 6, maximum length: 128.

Important rules:

- Email addresses are normalized to lowercase.
- Only active users can log in.
- Invalid email, inactive user, and invalid password all return the same generic authentication error.

Example request:

```
POST /auth/login
Content-Type: application/json
```

```
{
  "email": "user@example.com",
  "password": "password123"
}
```

Example success response:

```
{
  "accessToken": "<jwt>",
  "tokenType": "Bearer",
  "user": {
    "id": "665000000000000000000001",
    "email": "user@example.com",
    "role": "user"
  }
}
```

Example authentication failure:

```
{
  "message": "Invalid credentials",
  "error": "Unauthorized",
  "statusCode": 401
}
```

4.3 GET /auth/me

Access: Authenticated user or administrator

Description: Returns the identity stored in the current access token.

Headers:

```
Authorization: Bearer <accessToken>
```

Request body: None.

Important rules:

- The endpoint requires a valid Bearer token.
- Expired or invalid tokens are rejected.
- The response describes the authenticated token identity.

Example request:

```
GET /auth/me
Authorization: Bearer <accessToken>
```

Example success response:

```
{
  "sub": "665000000000000000000001",
  "email": "user@example.com",
  "role": "user"
}
```

4.4 Authentication Response Shape

Both POST /auth/register and POST /auth/login return the same response shape:

```
{
  "accessToken": "string",
  "tokenType": "Bearer",
  "user": {
    "id": "string",
    "email": "string",
    "role": "user | admin"
  }
}
```

4.5 Current User Token Shape

Authenticated requests identify the current user using the token payload:

```
{
  "sub": "string",
  "email": "string",
  "role": "user | admin"
}
```

The sub field contains the user identifier.

5 User Endpoints

User endpoints cover two areas:

- self-service account operations for the currently authenticated user,
- administrator operations for managing users.

Public account registration is documented in the authentication section. The endpoints in this section require authentication.

5.1 User Response Shape

Most user endpoints return the following response shape:

```
{
  "id": "66500000000000000000000001",
  "email": "user@example.com",
  "role": "user",
  "status": "active",
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:00:00.000Z"
}
```

Soft-deleted users may also include:

```
{
  "deletedAt": "2026-05-30T10:00:00.000Z"
}
```

5.2 GET /users/me

Access: Authenticated user or administrator

Description: Returns the currently authenticated active user.

Headers:

```
Authorization: Bearer <accessToken>
```

Request body: None.

Important rules:

- The endpoint uses the current access token to identify the user.
- Deleted users are treated as not found.

Example request:

```
GET /users/me
Authorization: Bearer <accessToken>
```

Example success response:

```
{
  "id": "66500000000000000000000001",
  "email": "user@example.com",
  "role": "user",
  "status": "active",
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:00:00.000Z"
}
```

5.3 PATCH /users/me

Access: Authenticated user or administrator

Description: Updates the currently authenticated user's email or password.

Headers:

```
Authorization: Bearer <accessToken>  
Content-Type: application/json
```

Request body:

```
{
  "email": "new-email@example.com",
  "password": "newPassword123",
  "currentPassword": "oldPassword123"
}
```

Body fields:

- **email** — optional string, must be a valid email address, maximum length: 254.
- **password** — optional string, minimum length: 6, maximum length: 128.
- **currentPassword** — required when changing email or password, minimum length: 6, maximum length: 128.

Important rules:

- At least one editable field must be provided.
- The current password is required.
- The current password must match the user's existing password.
- Email addresses are normalized to lowercase.
- Email addresses must be unique.
- This endpoint does not allow changing role or status.

Example request:

```
PATCH /users/me
Authorization: Bearer <accessToken>
Content-Type: application/json
```

```
{
  "email": "new-email@example.com",
  "currentPassword": "oldPassword123"
}
```

Example success response:

```
{
  "id": "665000000000000000000001",
  "email": "new-email@example.com",
  "role": "user",
  "status": "active",
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:05:00.000Z"
}
```

5.4 DELETE /users/me

Access: Authenticated regular user

Description: Deactivates the currently authenticated regular user's own account.

Headers:

```
Authorization: Bearer <accessToken>
```

Request body: None.

Important rules:

- Only users with the **user** role can deactivate their own account.
- Administrators cannot deactivate their own account through this endpoint.
- Deactivation is implemented as soft deletion.
- Successful deletion returns 204 No Content.

Example request:

```
DELETE /users/me
Authorization: Bearer <accessToken>
```

5.5 POST /users

Access: Administrator

Description: Creates a user account. This endpoint can be used by administrators to create either regular users or administrators.

Headers:

```
Authorization: Bearer <adminAccessToken>
Content-Type: application/json
```

Request body:

```
{
  "email": "admin-created-user@example.com",
  "password": "password123",
  "role": "user"
}
```

Body fields:

- **email** — required string, must be a valid email address, maximum length: 254.
- **password** — required string, minimum length: 6, maximum length: 128.
- **role** — optional enum: `user`, `admin`. Defaults to `user` when omitted.

Important rules:

- Only administrators can call this endpoint.
- Email addresses are normalized to lowercase.
- Email addresses must be unique.
- Created users start with status `active`.

Example request:

```
POST /users
Authorization: Bearer <adminAccessToken>
Content-Type: application/json
```

```
{
  "email": "admin-created-user@example.com",
  "password": "password123",
  "role": "user"
}
```

Example success response:

```
{
  "id": "665000000000000000000002",
  "email": "admin-created-user@example.com",
  "role": "user",
  "status": "active",
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:00:00.000Z"
}
```

5.6 GET /users

Access: Administrator

Description: Returns a paginated list of users.

Headers:

```
Authorization: Bearer <adminAccessToken>
```

Query parameters:

- **page** — optional positive integer, default: 1.
- **limit** — optional positive integer, default: 20, maximum: 100.
- **role** — optional enum: `user`, `admin`.
- **search** — optional string used to search by email.

Example request:

```
GET /users?page=1&limit=20&role=user&search=example
Authorization: Bearer <adminAccessToken>
```

Example success response:

```
{
  "items": [
    {
      "id": "665000000000000000000002",
      "email": "admin-created-user@example.com",
      "role": "user",
      "status": "active",
      "createdAt": "2026-05-30T10:00:00.000Z",
      "updatedAt": "2026-05-30T10:00:00.000Z"
    }
  ],
  "page": 1,
  "limit": 20,
  "total": 1,
  "totalPages": 1
}
```

5.7 GET /users/:id

Access: Administrator

Description: Returns a specific user by identifier.

Path parameters:

- id — user ObjectId.

Headers:

```
Authorization: Bearer <adminAccessToken>
```

Example request:

```
GET /users/665000000000000000000002
Authorization: Bearer <adminAccessToken>
```

Example success response:

```
{
  "id": "665000000000000000000002",
  "email": "admin-created-user@example.com",
  "role": "user",
  "status": "active",
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:00:00.000Z"
}
```

5.8 PATCH /users/:id

Access: Administrator

Description: Updates a user account by identifier.

Path parameters:

- id — user ObjectId.

Headers:

```
Authorization: Bearer <adminAccessToken>
Content-Type: application/json
```

Request body:

```
{
  "email": "updated-user@example.com",
  "password": "newPassword123",
  "role": "admin",
  "status": "active"
}
```

Body fields:

- email — optional string, must be a valid email address, maximum length: 254.
- password — optional string, minimum length: 6, maximum length: 128.
- role — optional enum: user, admin.
- status — optional enum: active, deleted.

Important rules:

- At least one editable field must be provided.
- Email addresses are normalized to lowercase.
- Email addresses must be unique.
- The system prevents removing or demoting the last active administrator.

Example request:

```
PATCH /users/66500000000000000000000000000002
Authorization: Bearer <adminAccessToken>
Content-Type: application/json
```

```
{
  "role": "admin"
}
```

Example success response:

```
{
  "id": "665000000000000000000002",
  "email": "admin-created-user@example.com",
  "role": "admin",
  "status": "active",
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:10:00.000Z"
}
```

5.9 DELETE /users/:id

Access: Administrator

Description: Soft-deletes a user account by identifier.

Path parameters:

- id — user ObjectId.

Headers:

```
Authorization: Bearer <adminAccessToken>
```

Request body: None.

Important rules:

- The endpoint changes the user status to `deleted`.
- The system prevents deleting the last active administrator.
- Successful deletion returns 204 No Content.

Example request:

```
DELETE /users/665000000000000000000002
Authorization: Bearer <adminAccessToken>
```

6 Category Endpoints

Category endpoints expose the product category hierarchy. Categories are nested, can define product attribute definitions, and can inherit attributes from ancestor categories.

Categories are public for reading, while creation, update, and archiving require administrator access.

6.1 Category Response Shape

Most category endpoints return a category object with the following structure:

```
{
  "id": "665000000000000000000001",
  "name": "Smart Toasters",
  "slug": "smart-toasters",
  "parentId": "665000000000000000000000",
  "ancestorIds": [
    "664000000000000000000000"
  ],
  "level": 1,
  "description": "High-tech bread scorching appliances.",
  "status": "active",
  "attributeDefinitions": [
    {
      "key": "burn_pattern",
      "label": "Burn Pattern",
      "type": "string",
      "isRequired": true,
      "allowedValues": [
        "sad_face",
        "qr_code",
        "obama"
      ]
    }
  ],
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:00:00.000Z"
}
```

Root categories do not have `parentId`. They have `ancestorIds` as an empty array and `level` equal to 0.

6.2 Category Attribute Definition Shape

Category attributes describe which product attribute values may be provided for products assigned to a category.

```
{
  "key": "slices",
  "label": "Slices Capacity",
  "type": "number",
  "isRequired": true,
  "min": 1,
  "max": 8,
  "unit": "slices"
}
```

Attribute definition fields:

- **key** — required string, valid identifier using lowercase letters, numbers, and under-scores. It must start with a letter.
- **label** — required string, human-readable field label.
- **type** — required enum: `string`, `number`, `boolean`, `string_array`.
- **isRequired** — required boolean. If true, products in this category branch must provide a value for this attribute.
- **allowedValues** — optional string array. Useful for select and multi-select fields.
- **min** — optional number. Useful for numeric attributes.
- **max** — optional number. Useful for numeric attributes.
- **unit** — optional string. Useful for displaying numeric attributes.

Important rules:

- Attribute keys must be unique within a category.
- A child category cannot redefine an attribute key inherited from an ancestor category.
- If both **min** and **max** are provided, **min** cannot be greater than **max**.
- Products store attribute values, while categories define which attributes are allowed.

6.3 GET /categories

Access: Public

Description: Returns a paginated list of active categories.

Query parameters:

- **page** — optional positive integer, default: 1.
- **limit** — optional positive integer, default: 20, maximum: 100.

- **search** — optional string used to search categories.
- **status** — optional enum: `active`, `archived`. On public endpoints, only active categories are returned.
- **parentId** — optional category `ObjectId` or `root`. Use `root` to request root categories only.

Example request:

```
GET /categories?page=1&limit=20
```

Example root category request:

```
GET /categories?parentId=root
```

Example child category request:

```
GET /categories?parentId=66500000000000000000000000000001
```

Example success response:

```
{
  "items": [
    {
      "id": "66500000000000000000000000000001",
      "name": "RTV",
      "slug": "rtv",
      "ancestorIds": [],
      "level": 0,
      "description": "Electronics and home media products",
      "status": "active",
      "attributeDefinitions": [],
      "createdAt": "2026-05-30T10:00:00.000Z",
      "updatedAt": "2026-05-30T10:00:00.000Z"
    }
  ],
  "page": 1,
  "limit": 20,
  "total": 1,
  "totalPages": 1
}
```

6.4 GET /categories/root

Access: Public

Description: Returns active root categories. Root categories are categories without a parent category.

Request body: None.

Example request:

```
GET /categories/root
```

Example success response:

```
[
  {
    "id": "66500000000000000000000000000001",
    "name": "RTV",
    "slug": "rtv",
    "ancestorIds": [],
    "level": 0,
    "description": "Electronics and home media products",
    "status": "active",
    "attributeDefinitions": [],
    "createdAt": "2026-05-30T10:00:00.000Z",
    "updatedAt": "2026-05-30T10:00:00.000Z"
  }
]
```

6.5 GET /categories/tree

Access: Public

Description: Returns all active categories as a nested tree.

Request body: None.

Important rules:

- Only active categories are returned.
- Each category may contain a `children` array.
- This endpoint is useful for category menus and category selection trees.

Example request:

```
GET /categories/tree
```

Example success response:

```
[
  {
    "id": "66500000000000000000000000000001",
    "name": "RTV",
    "slug": "rtv",
    "ancestorIds": [],
    "level": 0,
    "description": "Electronics and home media products",
    "status": "active",
    "attributeDefinitions": [],
    "createdAt": "2026-05-30T10:00:00.000Z",
    "updatedAt": "2026-05-30T10:00:00.000Z",
    "children": [
      {
        "id": "66500000000000000000000000000002",
        "name": "Smart Toasters",
        "slug": "smart-toasters",
        "parentId": "66500000000000000000000000000001",
        "ancestorIds": [
          "66500000000000000000000000000001"
        ],
        "level": 1,
        "description": "High-tech bread scorching appliances.",
        "status": "active",
        "attributeDefinitions": [],
        "createdAt": "2026-05-30T10:00:00.000Z",
        "updatedAt": "2026-05-30T10:00:00.000Z",
        "children": []
      }
    ]
  }
]
```

6.6 GET /categories/:id

Access: Public

Description: Returns one active category by identifier.

Path parameters:

- id — category ObjectId.

Example request:

```
GET /categories/66500000000000000000000000000001
```

Example success response:

```
{
  "id": "66500000000000000000000000000001",
  "name": "RTV",
  "slug": "rtv",
  "ancestorIds": [],
  "level": 0,
  "description": "Electronics and home media products",
  "status": "active",
  "attributeDefinitions": [],
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:00:00.000Z"
}
```

6.7 GET /categories/slug/:slug

Access: Public

Description: Returns one active category by slug.

Path parameters:

- slug — category slug.

Example request:

```
GET /categories/slug/smart-toasters
```

6.8 GET /categories/:id/children

Access: Public

Description: Returns direct active children of the selected category.

Path parameters:

- id — parent category ObjectId.

Example request:

```
GET /categories/66500000000000000000000000000001/children
```

Example success response:

```
[
  {
    "id": "665000000000000000000002",
    "name": "Smart Toasters",
    "slug": "smart-toasters",
    "parentId": "665000000000000000000001",
    "ancestorIds": [
      "665000000000000000000001"
    ],
    "level": 1,
    "description": "High-tech bread scorching appliances.",
    "status": "active",
    "attributeDefinitions": [],
    "createdAt": "2026-05-30T10:00:00.000Z",
    "updatedAt": "2026-05-30T10:00:00.000Z"
  }
]
```

6.9 GET /categories/:id/tree

Access: Public

Description: Returns one active category with its active descendants as a subtree.

Path parameters:

- `id` — root category ObjectId for the subtree.

Example request:

```
GET /categories/665000000000000000000001/tree
```

Important rules:

- This endpoint is useful for rendering a category branch.
- Only active categories are returned.

6.10 GET /categories/:id/attributes

Access: Public

Description: Returns inherited attribute definitions for a category. The response includes attributes defined by ancestor categories and by the selected category.

Path parameters:

- `id` — category ObjectId.

Important rules:

- This endpoint should be used by the frontend when building product creation or product update forms.
- Returned attributes are already merged in inheritance order.
- Products assigned to this category may only provide values for these attributes.

Example request:

```
GET /categories/6650000000000000000002/attributes
```

Example success response:

```
{
  "categoryId": "6650000000000000000002",
  "attributes": [
    {
      "key": "brand",
      "label": "Brand",
      "type": "string",
      "isRequired": true,
      "allowedValues": [
        "Samsung",
        "Sony",
        "Logitech"
      ]
    },
    {
      "key": "burn_pattern",
      "label": "Burn Pattern",
      "type": "string",
      "isRequired": true,
      "allowedValues": [
        "sad_face",
        "qr_code",
        "obama"
      ]
    },
    {
      "key": "slices",
      "label": "Slices Capacity",
      "type": "number",
      "isRequired": true,
      "min": 1,
      "max": 8
    }
  ]
}
```

6.11 GET /categories/admin

Access: Administrator

Description: Returns a paginated category list for administrators. Unlike the public list, this endpoint may include archived categories.

Headers:

```
Authorization: Bearer <adminAccessToken>
```

Query parameters:

- `page` — optional positive integer, default: 1.
- `limit` — optional positive integer, default: 20, maximum: 100.
- `search` — optional string used to search categories.
- `status` — optional enum: `active`, `archived`.
- `parentId` — optional category `ObjectId` or `root`.

Example request:

```
GET /categories/admin?status=archived&page=1&limit=20
Authorization: Bearer <adminAccessToken>
```

6.12 GET /categories/admin/:id

Access: Administrator

Description: Returns one category by identifier, including archived categories.

Path parameters:

- `id` — category `ObjectId`.

Headers:

```
Authorization: Bearer <adminAccessToken>
```

Example request:

```
GET /categories/admin/66500000000000000000000000000001
Authorization: Bearer <adminAccessToken>
```

6.13 POST /categories

Access: Administrator

Description: Creates a category. If `parentId` is omitted, a root category is created. If `parentId` is provided, the category becomes a child of the selected active parent category.

Headers:

```
Authorization: Bearer <adminAccessToken>
Content-Type: application/json
```

Request body:

```
{
  "name": "Smart Toasters",
  "slug": "smart-toasters",
  "parentId": "665000000000000000000001",
  "description": "High-tech bread scorching appliances.",
  "status": "active",
  "attributeDefinitions": [
    {
      "key": "burn_pattern",
      "label": "Burn Pattern",
      "type": "string",
      "isRequired": true,
      "allowedValues": [
        "sad_face",
        "qr_code",
        "obama"
      ]
    },
    {
      "key": "slices",
      "label": "Slices Capacity",
      "type": "number",
      "isRequired": true,
      "min": 1,
      "max": 8
    }
  ]
}
```

Body fields:

- **name** — required string.
- **slug** — optional slug string. If omitted, it is generated from **name**.
- **parentId** — optional parent category ObjectId.
- **description** — optional string.
- **status** — optional enum: **active**. Creating a category as **archived** is rejected.
- **attributeDefinitions** — optional array of category attribute definitions.

Important rules:

- Only administrators can create categories.
- A new category cannot be created as **archived**.

- If `parentId` is omitted, a root category is created.
- If `parentId` is provided, the parent category must exist and be active.
- The maximum category depth is limited.
- Attribute keys must be unique within the new category.
- A child category cannot define an attribute key already inherited from an ancestor.
- Category slugs must be unique.

Example root category request:

```
POST /categories
Authorization: Bearer <adminAccessToken>
Content-Type: application/json
```

```
{
  "name": "RTV",
  "slug": "rtv",
  "description": "Electronics and home media products",
  "status": "active",
  "attributeDefinitions": [
    {
      "key": "brand",
      "label": "Brand",
      "type": "string",
      "isRequired": true,
      "allowedValues": [
        "Samsung",
        "Sony",
        "Logitech"
      ]
    }
  ]
}
```

Example child category request:

```
POST /categories
Authorization: Bearer <adminAccessToken>
Content-Type: application/json
```

```
{
  "name": "Smart Toasters",
  "slug": "smart-toasters",
  "parentId": "665000000000000000000001",
  "description": "High-tech bread scorching appliances.",
  "status": "active",
  "attributeDefinitions": [
    {
      "key": "slices",
      "label": "Slices Capacity",
      "type": "number",
      "isRequired": true,
      "min": 1,
      "max": 8
    }
  ]
}
```

6.14 PATCH /categories/:id

Access: Administrator

Description: Updates category metadata, status, or attribute definitions.

Path parameters:

- id — category ObjectId.

Headers:

```
Authorization: Bearer <adminAccessToken>
Content-Type: application/json
```

Request body:

```
{
  "name": "Updated Category Name",
  "slug": "updated-category-name",
  "description": "Updated category description.",
  "status": "active",
  "attributeDefinitions": [
    {
      "key": "slices",
      "label": "Slices Capacity",
      "type": "number",
      "isRequired": true,
      "min": 1,
      "max": 8
    }
  ]
}
```

Body fields:

- name — optional string.
- slug — optional slug string.
- description — optional string.
- status — optional enum: active, archived.
- attributeDefinitions — optional array of category attribute definitions.

Important rules:

- At least one editable field must be provided.
- parentId cannot be changed by this endpoint.
- Category slugs must be unique.
- Attribute keys must be unique within the category.
- A category cannot define an attribute key already inherited from an ancestor.
- Updating status to archived sets archivedAt.
- Updating status away from archived removes archivedAt.

Example request:

```
PATCH /categories/665000000000000000000002
Authorization: Bearer <adminAccessToken>
Content-Type: application/json
```

```
{
  "description": "Updated category description."
}
```

6.15 DELETE /categories/:id

Access: Administrator

Description: Archives a category.

Path parameters:

- id — category ObjectId.

Headers:

```
Authorization: Bearer <adminAccessToken>
```

Request body: None.

Important rules:

- The endpoint archives the category instead of hard-deleting it.
- Categories with child categories cannot be archived.
- Archiving sets category status to **archived**.
- Successful archive returns 204 No Content.

Example request:

```
DELETE /categories/665000000000000000000002
Authorization: Bearer <adminAccessToken>
```

7 Product Endpoints

Product endpoints expose product listing, product details, and administrator product management.

Public product endpoints return only active products. Administrator endpoints can access products regardless of status.

Products belong to categories. Product-specific attributes are stored as `attributeValues`, but the allowed attribute keys and constraints are defined by the selected category and its ancestors.

The public product list endpoint is optimized for frontend product card views. It returns only the data needed for a listing view: product name, slug, description, price, and primary image. Full product data is still available through product detail endpoints and administrator endpoints.

7.1 Product List Item Response Shape

The public product listing endpoint returns compact product items.

```
{
  "id": "665000000000000000000010",
  "name": "Smart Toaster 3000",
  "slug": "smart-toaster-3000",
  "description": "A toaster with advanced burn pattern support.",
  "price": {
    "amount": 24999,
    "currency": "PLN"
  },
  "image": {
    "url": "https://example.com/toaster.jpg",
    "alt": "Smart Toaster 3000"
  }
}
```

If the product has no image, the `image` field is `null`.

```
{
  "image": null
}
```

Important rules:

- This compact shape is used by `GET /products`.
- The endpoint is intended for product listing and product card views.
- It does not include stock, category, attributes, status, rating statistics, or timestamps.
- Full product data should be requested from `GET /products/:id` or `GET /products/slug/:slug`.

7.2 Product Response Shape

Product detail and administrator endpoints return the full product shape:

```
{
  "id": "66500000000000000000000010",
  "name": "Smart Toaster 3000",
  "slug": "smart-toaster-3000",
  "description": "A toaster with advanced burn pattern support.",
  "price": {
    "amount": 24999,
    "currency": "PLN"
  },
  "stock": 10,
  "categoryId": "66500000000000000000000002",
  "images": [
    {
      "url": "https://example.com/toaster.jpg",
      "alt": "Smart Toaster 3000",
      "isPrimary": true
    }
  ],
  "attributeValues": {
    "brand": "Samsung",
    "warranty_months": 24,
    "burn_pattern": "qr_code",
    "slices": 4
  },
  "status": "active",
  "averageRating": 4.5,
  "reviewCount": 12,
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:00:00.000Z"
}
```

Archived products may also include:

```
{
  "archivedAt": "2026-05-30T10:00:00.000Z"
}
```

7.3 Product Price Shape

Product prices use minor currency units.

```
{
  "amount": 24999,
  "currency": "PLN"
}
```

This represents 249.99 PLN.

Price fields:

- `amount` — required integer, minimum: 0.
- `currency` — required enum: PLN, EUR, USD.

7.4 Product Image Shape

```
{
  "url": "https://example.com/product.jpg",
  "alt": "Product image",
  "isPrimary": true
}
```

Image fields:

- `url` — required URL string. The URL must use `http` or `https`.
- `alt` — optional string.
- `isPrimary` — required boolean.

Important rules:

- A product can have at most one primary image.
- If images are provided but none is marked as primary, the backend marks the first image as primary.
- Public product listing returns only the primary image as a compact `image` object.
- Full product detail endpoints return the full `images` array.

7.5 Product Attribute Values

Products store attribute values in the `attributeValues` object.

```
{
  "attributeValues": {
    "brand": "Samsung",
    "warranty_months": 24,
    "burn_pattern": "qr_code",
    "slices": 4
  }
}
```

Important rules:

- A product must belong to an active category.
- Allowed product attribute keys come from the selected category and its ancestors.
- Required category attributes must be present in `attributeValues`.
- Attribute values must match their category-defined type.
- String attributes with `allowedValues` must use one of those values.
- Number attributes must satisfy optional `min` and `max` constraints.
- Boolean attributes must be boolean values.
- `string_array` attributes must be arrays of strings.

Before creating or editing a product, the frontend should usually call:

```
GET /categories/:id/attributes
```

This returns the inherited attribute definitions needed to build the product form.

7.6 GET /products

Access: Public

Description: Returns a paginated compact list of active products for product listing and product card views.

Query parameters:

- `page` — optional positive integer, default: 1.
- `limit` — optional positive integer, default: 20, maximum: 100.
- `search` — optional string used to search products.
- `status` — optional enum: `draft`, `active`, `archived`. Public listing still returns only active products.
- `minPrice` — optional integer, minimum price amount in minor currency units.
- `maxPrice` — optional integer, maximum price amount in minor currency units.

- `inStockOnly` — optional boolean: `true` or `false`.
- `categoryId` — optional category `ObjectId`.
- `includeSubcategories` — optional boolean. When used with `categoryId`, includes products from descendant categories.

Important rules:

- Public listing returns active products only.
- The response uses a compact product list item shape.
- The response includes only product card data: `id`, `name`, `slug`, `description`, `price`, and `image`.
- Stock, category, attributes, status, ratings, and timestamps are intentionally omitted from this endpoint.
- `minPrice` cannot be greater than `maxPrice`.
- If `categoryId` is provided, the category must exist and be active.
- If `includeSubcategories=true`, products from descendant categories are included.
- Full product data should be requested from `GET /products/:id` or `GET /products/slug/:slug`.

Example request:

```
GET /products?page=1&limit=20
```

Example category branch request:

```
GET /products?categoryId=665000000000000000000002&includeSubcategories=true
```

Example filtered request:

```
GET /products?search=toaster&minPrice=10000&maxPrice=30000&inStockOnly=true
```

Example success response:

```
{
  "items": [
    {
      "id": "66500000000000000000010",
      "name": "Smart Toaster 3000",
      "slug": "smart-toaster-3000",
      "description": "A toaster with advanced burn pattern support.",
      "price": {
        "amount": 24999,
        "currency": "PLN"
      },
      "image": {
        "url": "https://example.com/toaster.jpg",
        "alt": "Smart Toaster 3000"
      }
    }
  ],
  "page": 1,
  "limit": 20,
  "total": 1,
  "totalPages": 1
}
```

7.7 GET /products/:id

Access: Public

Description: Returns one active product by identifier. This endpoint returns the full public product details.

Path parameters:

- id — product ObjectId.

Example request:

```
GET /products/66500000000000000000010
```

Example success response:

```
{
  "id": "66500000000000000000000010",
  "name": "Smart Toaster 3000",
  "slug": "smart-toaster-3000",
  "description": "A toaster with advanced burn pattern support.",
  "price": {
    "amount": 24999,
    "currency": "PLN"
  },
  "stock": 10,
  "categoryId": "66500000000000000000000002",
  "images": [],
  "attributeValues": {
    "brand": "Samsung",
    "burn_pattern": "qr_code",
    "slices": 4
  },
  "status": "active",
  "averageRating": 0,
  "reviewCount": 0,
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:00:00.000Z"
}
```

7.8 GET /products/slug/:slug

Access: Public

Description: Returns one active product by slug. This endpoint returns the full public product details.

Path parameters:

- slug — product slug.

Example request:

```
GET /products/slug/smart-toaster-3000
```

7.9 GET /products/admin

Access: Administrator

Description: Returns a paginated product list for administrators. Unlike the public listing, this endpoint can include draft and archived products and returns full product data.

Headers:

```
Authorization: Bearer <adminAccessToken>
```

Query parameters:

- **page** — optional positive integer, default: 1.
- **limit** — optional positive integer, default: 20, maximum: 100.
- **search** — optional string used to search products.
- **status** — optional enum: **draft**, **active**, **archived**.
- **minPrice** — optional integer, minimum price amount in minor currency units.
- **maxPrice** — optional integer, maximum price amount in minor currency units.
- **inStockOnly** — optional boolean: **true** or **false**.
- **categoryId** — optional category **ObjectId**.
- **includeSubcategories** — optional boolean. When used with **categoryId**, includes products from descendant categories.

Example request:

```
GET /products/admin?status=draft&page=1&limit=20
Authorization: Bearer <adminAccessToken>
```

7.10 GET /products/admin/:id

Access: Administrator

Description: Returns one product by identifier, including draft or archived products.

Path parameters:

- `id` — product `ObjectId`.

Headers:

```
Authorization: Bearer <adminAccessToken>
```

Example request:

```
GET /products/admin/6650000000000000000010
Authorization: Bearer <adminAccessToken>
```

7.11 POST /products

Access: Administrator

Description: Creates a product.

Headers:

```
Authorization: Bearer <adminAccessToken>  
Content-Type: application/json
```

Request body:

```

{
  "name": "Smart Toaster 3000",
  "slug": "smart-toaster-3000",
  "description": "A toaster with advanced burn pattern support.",
  "price": {
    "amount": 24999,
    "currency": "PLN"
  },
  "stock": 10,
  "categoryId": "665000000000000000000002",
  "images": [
    {
      "url": "https://example.com/toaster.jpg",
      "alt": "Smart Toaster 3000",
      "isPrimary": true
    }
  ],
  "attributeValues": {
    "brand": "Samsung",
    "warranty_months": 24,
    "burn_pattern": "qr_code",
    "slices": 4
  },
  "status": "active"
}

```

Body fields:

- **name** — required string.
- **slug** — optional slug string. If omitted, it is generated from **name**.
- **description** — required string.
- **price** — required object containing **amount** and **currency**.
- **stock** — required integer, minimum: 0.
- **categoryId** — required active category **ObjectId**.
- **images** — optional array of product images.
- **attributeValues** — optional object containing product attribute values. Required category attributes must still be present.
- **status** — optional enum: **draft**, **active**. Defaults to **draft**. Creating a product as **archived** is rejected.

Important rules:

- Only administrators can create products.
- New products cannot be created as **archived**.

- Product slugs must be unique.
- The selected category must exist and be active.
- `attributeValues` must match the inherited attributes for the selected category.
- Required inherited category attributes must be present.
- Only one image can be primary.
- If images are provided and none is primary, the first image is made primary.

Example request:

```
POST /products
Authorization: Bearer <adminAccessToken>
Content-Type: application/json
```

```
{
  "name": "Smart Toaster 3000",
  "description": "A toaster with advanced burn pattern support.",
  "price": {
    "amount": 24999,
    "currency": "PLN"
  },
  "stock": 10,
  "categoryId": "665000000000000000000002",
  "images": [],
  "attributeValues": {
    "brand": "Samsung",
    "burn_pattern": "qr_code",
    "slices": 4
  },
  "status": "active"
}
```

7.12 PATCH /products/:id

Access: Administrator

Description: Updates a product.

Path parameters:

- `id` — product `ObjectId`.

Headers:

```
Authorization: Bearer <adminAccessToken>
Content-Type: application/json
```

Request body:

```
{
  "name": "Updated Smart Toaster 3000",
  "slug": "updated-smart-toaster-3000",
  "description": "Updated product description.",
  "price": {
    "amount": 25999,
    "currency": "PLN"
  },
  "stock": 15,
  "categoryId": "665000000000000000000002",
  "images": [
    {
      "url": "https://example.com/toaster-updated.jpg",
      "alt": "Updated Smart Toaster 3000",
      "isPrimary": true
    }
  ],
  "attributeValues": {
    "brand": "Samsung",
    "warranty_months": 24,
    "burn_pattern": "sad_face",
    "slices": 4
  },
  "status": "active"
}
```

Body fields:

- name — optional string.
- slug — optional slug string.
- description — optional string.
- price — optional object containing amount and currency.
- stock — optional integer, minimum: 0.
- categoryId — optional active category ObjectId.
- images — optional array of product images.
- attributeValues — optional object containing product attribute values.
- status — optional enum: draft, active, archived.

Important rules:

- At least one editable field must be provided.
- Product slugs must be unique.
- If categoryId is changed, the new category must exist and be active.

- The effective `attributeValues` must match the effective category.
- If `attributeValues` is omitted, existing product attribute values are validated against the effective category.
- Only one image can be primary.
- If images are provided and none is primary, the first image is made primary.
- Updating status to `archived` sets `archivedAt`.
- Updating status away from `archived` removes `archivedAt`.

Example request:

```
PATCH /products/66500000000000000000010
Authorization: Bearer <adminAccessToken>
Content-Type: application/json
```

```
{
  "stock": 15,
  "status": "active"
}
```

7.13 DELETE /products/:id

Access: Administrator

Description: Archives a product.

Path parameters:

- `id` — product `ObjectId`.

Headers:

```
Authorization: Bearer <adminAccessToken>
```

Request body: None.

Important rules:

- The endpoint archives the product instead of hard-deleting it.
- Archiving sets product status to `archived`.
- Successful archive returns 204 No Content.

Example request:

```
DELETE /products/66500000000000000000010
Authorization: Bearer <adminAccessToken>
```

8 Cart Endpoints

Cart endpoints allow an authenticated regular user to view and manage their shopping cart.

The cart is user-specific. Each regular user has at most one cart. Cart endpoints are available only to users with the `user` role.

8.1 Cart Response Shape

Cart endpoints return the following response shape:

```
{
  "id": "665000000000000000000020",
  "userId": "665000000000000000000001",
  "items": [
    {
      "productId": "665000000000000000000010",
      "quantity": 2,
      "addedAt": "2026-05-30T10:00:00.000Z",
      "updatedAt": "2026-05-30T10:05:00.000Z",
      "isAvailable": true,
      "lineTotalAmount": 49998,
      "product": {
        "id": "665000000000000000000010",
        "name": "Smart Toaster 3000",
        "slug": "smart-toaster-3000",
        "price": {
          "amount": 24999,
          "currency": "PLN"
        },
        "stock": 10,
        "imageUrl": "https://example.com/toaster.jpg"
      }
    }
  ],
  "totalQuantity": 2,
  "totals": [
    {
      "currency": "PLN",
      "amount": 49998
    }
  ],
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:05:00.000Z"
}
```

If the user does not have a cart yet, the response uses an empty cart shape:

```
{
  "id": null,
  "userId": "66500000000000000000000000000001",
  "items": [],
  "totalQuantity": 0,
  "totals": [],
  "createdAt": null,
  "updatedAt": null
}
```

8.2 Cart Item Response Shape

Each cart item contains cart-specific fields and a product summary.

```
{
  "productId": "66500000000000000000000000000010",
  "quantity": 2,
  "addedAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:05:00.000Z",
  "isAvailable": true,
  "lineTotalAmount": 49998,
  "product": {
    "id": "66500000000000000000000000000010",
    "name": "Smart Toaster 3000",
    "slug": "smart-toaster-3000",
    "price": {
      "amount": 24999,
      "currency": "PLN"
    },
    "stock": 10,
    "imageUrl": "https://example.com/toaster.jpg"
  }
}
```

Important rules:

- `lineTotalAmount` is calculated as product price amount multiplied by item quantity.
- If the product is no longer active or available, `isAvailable` is `false`.
- Unavailable items have `lineTotalAmount` set to `null`.
- Unavailable items have `product` set to `null`.
- Cart totals include only currently available active products.

8.3 GET /cart

Access: Authenticated regular user

Description: Returns the current user's cart.

Headers:

```
Authorization: Bearer <accessToken>
```

Request body: None.

Important rules:

- The endpoint returns the cart belonging to the authenticated user.
- If the user does not have a cart yet, an empty cart response is returned.
- Product data in the response reflects current active product data.
- Products that are no longer active are represented as unavailable cart items.

Example request:

```
GET /cart  
Authorization: Bearer <accessToken>
```

Example success response:

```
{
  "id": "66500000000000000000020",
  "userId": "665000000000000000000001",
  "items": [
    {
      "productId": "66500000000000000000010",
      "quantity": 2,
      "addedAt": "2026-05-30T10:00:00.000Z",
      "updatedAt": "2026-05-30T10:05:00.000Z",
      "isAvailable": true,
      "lineTotalAmount": 49998,
      "product": {
        "id": "66500000000000000000010",
        "name": "Smart Toaster 3000",
        "slug": "smart-toaster-3000",
        "price": {
          "amount": 24999,
          "currency": "PLN"
        },
        "stock": 10,
        "imageUrl": "https://example.com/toaster.jpg"
      }
    }
  ],
  "totalQuantity": 2,
  "totals": [
    {
      "currency": "PLN",
      "amount": 49998
    }
  ],
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:05:00.000Z"
}
```

8.4 POST /cart/items

Access: Authenticated regular user

Description: Adds a product to the current user's cart. If the product is already in the cart, the requested quantity is added to the existing quantity.

Headers:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```

Request body:

```
{
  "productId": "665000000000000000000010",
  "quantity": 2
}
```

Body fields:

- productId — required product ObjectId.
- quantity — required integer within the allowed cart item quantity range.

Important rules:

- The product must exist and be active.
- The product must be in stock.
- The final cart quantity cannot exceed current product stock.
- If the item already exists in the cart, this endpoint increments its quantity.
- If the user does not have a cart yet, the cart is created automatically.

Example request:

```
POST /cart/items
Authorization: Bearer <accessToken>
Content-Type: application/json
```

```
{
  "productId": "665000000000000000000010",
  "quantity": 2
}
```

Example success response:

```
{
  "id": "66500000000000000000020",
  "userId": "665000000000000000000001",
  "items": [
    {
      "productId": "66500000000000000000010",
      "quantity": 2,
      "addedAt": "2026-05-30T10:00:00.000Z",
      "updatedAt": "2026-05-30T10:00:00.000Z",
      "isAvailable": true,
      "lineTotalAmount": 49998,
      "product": {
        "id": "66500000000000000000010",
        "name": "Smart Toaster 3000",
        "slug": "smart-toaster-3000",
        "price": {
          "amount": 24999,
          "currency": "PLN"
        },
        "stock": 10
      }
    }
  ],
  "totalQuantity": 2,
  "totals": [
    {
      "currency": "PLN",
      "amount": 49998
    }
  ],
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:00:00.000Z"
}
```

8.5 PATCH /cart/items/:productId

Access: Authenticated regular user

Description: Sets the final quantity for a product in the current user's cart.

Path parameters:

- productId — product ObjectId.

Headers:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```

Request body:

```
{
  "quantity": 4
}
```

Body fields:

- quantity — required integer within the allowed cart item quantity range.

Important rules:

- The product must exist and be active.
- The product must be in stock.
- The requested quantity cannot exceed current product stock.
- This endpoint sets the final quantity instead of incrementing it.
- With the current implementation, if the item is not already in the cart, it is added with the requested quantity.
- If the user does not have a cart yet, the cart is created automatically.

Example request:

```
PATCH /cart/items/665000000000000000000010  
Authorization: Bearer <accessToken>  
Content-Type: application/json
```

```
{
  "quantity": 4
}
```

Example success response:

```
{
  "id": "66500000000000000000020",
  "userId": "665000000000000000000001",
  "items": [
    {
      "productId": "66500000000000000000010",
      "quantity": 4,
      "addedAt": "2026-05-30T10:00:00.000Z",
      "updatedAt": "2026-05-30T10:10:00.000Z",
      "isAvailable": true,
      "lineTotalAmount": 99996,
      "product": {
        "id": "66500000000000000000010",
        "name": "Smart Toaster 3000",
        "slug": "smart-toaster-3000",
        "price": {
          "amount": 24999,
          "currency": "PLN"
        },
        "stock": 10
      }
    }
  ],
  "totalQuantity": 4,
  "totals": [
    {
      "currency": "PLN",
      "amount": 99996
    }
  ],
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:10:00.000Z"
}
```

8.6 DELETE /cart/items/:productId

Access: Authenticated regular user

Description: Removes a product from the current user's cart.

Path parameters:

- productId — product ObjectId.

Headers:

```
Authorization: Bearer <accessToken>
```

Request body: None.

Important rules:

- The item is removed from the authenticated user's cart.
- Removing a missing item is treated as a successful no-op.
- Successful removal returns **204 No Content**.

Example request:

```
DELETE /cart/items/6650000000000000000010  
Authorization: Bearer <accessToken>
```

8.7 DELETE /cart

Access: Authenticated regular user

Description: Clears all items from the current user's cart.

Headers:

```
Authorization: Bearer <accessToken>
```

Request body: None.

Important rules:

- The cart document is kept, but its item list is cleared.
- Clearing an already empty or missing cart is treated as a successful no-op.
- Successful clearing returns **204 No Content**.

Example request:

```
DELETE /cart
Authorization: Bearer <accessToken>
```

9 Order Endpoints

Order endpoints allow authenticated regular users to create orders from their cart and view their own orders.

The current API exposes only user-owned order operations. There are no administrator order management endpoints in the current version.

Checkout supports both private and company purchases. Invoice-related data is stored directly as part of the order snapshot. The API does not create a separate invoice resource.

9.1 Order Response Shape

Most order endpoints return the following order shape:

```
{
  "id": "665000000000000000000030",
  "userId": "665000000000000000000001",
  "status": "pending_payment",
  "purchaseType": "private",
  "invoice": {
    "requested": false,
    "billingAddressSameAsShipping": true
  },
  "items": [
    {
      "productId": "665000000000000000000010",
      "productName": "Smart Toaster 3000",
      "productSlug": "smart-toaster-3000",
      "unitPriceAmount": 24999,
      "currency": "PLN",
      "quantity": 2,
      "imageUrl": "https://example.com/toaster.jpg"
    }
  ],
  "shippingAddress": {
    "fullName": "Jan Kowalski",
    "line1": "Example Street 12",
    "line2": "Apartment 4",
    "city": "Warsaw",
    "postalCode": "00-001",
    "country": "Poland",
    "phone": "+48123456789"
  },
  "totals": {
    "subtotalAmount": 49998,
    "shippingAmount": 0,
    "totalAmount": 49998,
    "currency": "PLN"
  },
  "paymentId": "665000000000000000000040",
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:00:00.000Z"
}
```

Paid orders may also include:

```
{
  "paidAt": "2026-05-30T10:05:00.000Z"
}
```

Cancelled orders may also include:

```
{
  "cancelledAt": "2026-05-30T10:05:00.000Z"
}
```

9.2 Purchase Type Values

Orders may use the following purchase type values:

- `private`,
- `company`.

If `purchaseType` is omitted during checkout, the backend treats the order as a private purchase.

9.3 Invoice Snapshot Shape

Invoice data is stored as an order snapshot. It is not a separate resource and does not create a legal invoice document by itself.

A private purchase without invoice request uses the following shape:

```
{
  "requested": false,
  "billingAddressSameAsShipping": true
}
```

A company purchase with invoice request may use the following shape:

```
{
  "requested": true,
  "billingAddressSameAsShipping": false,
  "billingAddress": {
    "fullName": "ACME Sp. z o.o.",
    "line1": "Business Street 1",
    "city": "Warsaw",
    "postalCode": "00-002",
    "country": "Poland"
  },
  "companyDetails": {
    "companyName": "ACME Sp. z o.o.",
    "taxId": "1234567890"
  }
}
```

Invoice fields:

- **requested** — boolean indicating whether invoice data was requested.
- **billingAddressSameAsShipping** — boolean indicating whether the shipping address should also be used as the billing address.
- **billingAddress** — optional billing address object. Required when invoice is requested and billing address is not the same as shipping address.
- **companyDetails** — optional company details object. Required for company purchases.

Company details fields:

- **companyName** — required string for company purchases.
- **taxId** — required string for company purchases.

Important rules:

- Company purchases require `invoice.requested` to be `true`.
- Company purchases require `invoice.companyDetails`.
- Private purchases cannot include company details.
- If invoice is not requested, billing address data is not required.
- If invoice is requested and `billingAddressSameAsShipping` is `true`, the shipping address is used as the billing address.
- If invoice is requested and `billingAddressSameAsShipping` is `false`, `billingAddress` is required.

9.4 Order Item Snapshot

Order items are stored as product snapshots. This means the order keeps selected product data from the time of checkout.

```
{
  "productId": "665000000000000000000010",
  "productName": "Smart Toaster 3000",
  "productSlug": "smart-toaster-3000",
  "unitPriceAmount": 24999,
  "currency": "PLN",
  "quantity": 2,
  "imageUrl": "https://example.com/toaster.jpg"
}
```

Important rules:

- `productId` points to the product.
- `productName`, `productSlug`, `unitPriceAmount`, `currency`, and `imageUrl` are copied into the order during checkout.
- Later product edits do not rewrite historical order item data.

9.5 Shipping Address Shape

Checkout requires a shipping address.

```
{
  "fullName": "Jan Kowalski",
  "line1": "Example Street 12",
  "line2": "Apartment 4",
  "city": "Warsaw",
  "postalCode": "00-001",
  "country": "Poland",
  "phone": "+48123456789"
}
```

Shipping address fields:

- `fullName` — required string.
- `line1` — required string.
- `line2` — optional string.
- `city` — required string.
- `postalCode` — required string.
- `country` — required string.
- `phone` — optional string.

The same address shape is also used for invoice billing addresses.

9.6 Order Totals Shape

```
{
  "subtotalAmount": 49998,
  "shippingAmount": 0,
  "totalAmount": 49998,
  "currency": "PLN"
}
```

Total fields:

- `subtotalAmount` — sum of item prices before shipping.
- `shippingAmount` — shipping cost.
- `totalAmount` — final amount to be paid.
- `currency` — order currency.

The current checkout implementation uses 0 as the shipping amount.

9.7 Order Status Values

Orders may use the following status values:

- `pending_payment`,
- `paid`,
- `payment_failed`,
- `cancelled`,
- `shipped`,
- `completed`.

9.8 POST /orders/checkout

Access: Authenticated regular user

Description: Creates an order from the current user's cart and creates a pending mock payment for that order.

Headers:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```

Minimal private purchase request body:

```
{
  "shippingAddress": {
    "fullName": "Jan Kowalski",
    "line1": "Example Street 12",
    "line2": "Apartment 4",
    "city": "Warsaw",
    "postalCode": "00-001",
    "country": "Poland",
    "phone": "+48123456789"
  }
}
```

Private purchase with invoice request body:

```
{
  "purchaseType": "private",
  "invoice": {
    "requested": true,
    "billingAddressSameAsShipping": true
  },
  "shippingAddress": {
    "fullName": "Jan Kowalski",
    "line1": "Example Street 12",
    "city": "Warsaw",
    "postalCode": "00-001",
    "country": "Poland"
  }
}
```

Company purchase request body:

```
{
  "purchaseType": "company",
  "invoice": {
    "requested": true,
    "billingAddressSameAsShipping": false,
    "billingAddress": {
      "fullName": "ACME Sp. z o.o.",
      "line1": "Business Street 1",
      "city": "Warsaw",
      "postalCode": "00-002",
      "country": "Poland"
    },
    "companyDetails": {
      "companyName": "ACME Sp. z o.o.",
      "taxId": "1234567890"
    }
  },
  "shippingAddress": {
    "fullName": "Warehouse Receiver",
    "line1": "Warehouse Street 5",
    "city": "Warsaw",
    "postalCode": "00-003",
    "country": "Poland",
    "phone": "+48123456789"
  }
}
```

Body fields:

- `purchaseType` — optional enum: `private`, `company`. Defaults to `private`.
- `invoice` — optional object containing invoice request data.
- `invoice.requested` — optional boolean. Defaults to `false`.
- `invoice.billingAddressSameAsShipping` — optional boolean. Defaults to `true`.
- `invoice.billingAddress` — optional address object. Required when invoice is requested and billing address is not the same as shipping address.
- `invoice.companyDetails` — optional company details object. Required for company purchases.
- `invoice.companyDetails.companyName` — required string for company purchases.
- `invoice.companyDetails.taxId` — required string for company purchases.
- `shippingAddress` — required object containing the shipping address.
- `shippingAddress.fullName` — required string.
- `shippingAddress.line1` — required string.

- `shippingAddress.line2` — optional string.
- `shippingAddress.city` — required string.
- `shippingAddress.postalCode` — required string.
- `shippingAddress.country` — required string.
- `shippingAddress.phone` — optional string.

Important rules:

- The user must have a non-empty cart.
- Every cart product must still exist and be active.
- Every cart item quantity must be available in stock.
- Checkout with multiple currencies is rejected.
- Product stock is decreased during checkout.
- The cart is cleared after successful checkout.
- The created order starts with status `pending_payment`.
- Checkout also creates a pending mock payment.
- Company purchases require invoice data and company details.
- Private purchases cannot include company details.
- Billing address may be copied from shipping address.
- The operation is transactional: order creation, payment creation, stock changes, and cart clearing are handled together.

Example request:

```
POST /orders/checkout
Authorization: Bearer <accessToken>
Content-Type: application/json
```

```
{
  "purchaseType": "company",
  "invoice": {
    "requested": true,
    "billingAddressSameAsShipping": false,
    "billingAddress": {
      "fullName": "ACME Sp. z o.o.",
      "line1": "Business Street 1",
      "city": "Warsaw",
      "postalCode": "00-002",
      "country": "Poland"
    },
    "companyDetails": {
      "companyName": "ACME Sp. z o.o.",
      "taxId": "1234567890"
    }
  },
  "shippingAddress": {
    "fullName": "Warehouse Receiver",
    "line1": "Warehouse Street 5",
    "city": "Warsaw",
    "postalCode": "00-003",
    "country": "Poland",
    "phone": "+48123456789"
  }
}
```

Example success response:

```
{
  "order": {
    "id": "66500000000000000000030",
    "userId": "66500000000000000000001",
    "status": "pending_payment",
    "purchaseType": "company",
    "invoice": {
      "requested": true,
      "billingAddressSameAsShipping": false,
      "billingAddress": {
        "fullName": "ACME Sp. z o.o.",
        "line1": "Business Street 1",
        "city": "Warsaw",
        "postalCode": "00-002",
        "country": "Poland"
      },
      "companyDetails": {
        "companyName": "ACME Sp. z o.o.",
        "taxId": "1234567890"
      }
    },
    "items": [
      {
        "productId": "66500000000000000000010",
        "productName": "Smart Toaster 3000",
        "productSlug": "smart-toaster-3000",
        "unitPriceAmount": 24999,
        "currency": "PLN",
        "quantity": 2,
        "imageUrl": "https://example.com/toaster.jpg"
      }
    ],
    "shippingAddress": {
      "fullName": "Warehouse Receiver",
      "line1": "Warehouse Street 5",
      "city": "Warsaw",
      "postalCode": "00-003",
      "country": "Poland",
      "phone": "+48123456789"
    },
    "totals": {
      "subtotalAmount": 49998,
      "shippingAmount": 0,
      "totalAmount": 49998,
      "currency": "PLN"
    },
    "paymentId": "66500000000000000000040",
  }
}
```

```
        "createdAt": "2026-05-30T10:00:00.000Z",
        "updatedAt": "2026-05-30T10:00:00.000Z"
    },
    "payment": {
        "id": "66500000000000000000000040",
        "orderId": "66500000000000000000000030",
        "userId": "66500000000000000000000001",
        "provider": "mock",
        "status": "pending",
        "amount": 49998,
        "currency": "PLN",
        "createdAt": "2026-05-30T10:00:00.000Z",
        "updatedAt": "2026-05-30T10:00:00.000Z"
    }
}
```

9.9 GET /orders/my

Access: Authenticated regular user

Description: Returns all orders owned by the currently authenticated user.

Headers:

```
Authorization: Bearer <accessToken>
```

Request body: None.

Important rules:

- The endpoint returns only orders belonging to the authenticated user.
- Orders are returned from newest to oldest.

Example request:

```
GET /orders/my
Authorization: Bearer <accessToken>
```

Example success response:

```
[
  {
    "id": "66500000000000000000030",
    "userId": "66500000000000000000001",
    "status": "pending_payment",
    "purchaseType": "private",
    "invoice": {
      "requested": false,
      "billingAddressSameAsShipping": true
    },
    "items": [
      {
        "productId": "66500000000000000000010",
        "productName": "Smart Toaster 3000",
        "productSlug": "smart-toaster-3000",
        "unitPriceAmount": 24999,
        "currency": "PLN",
        "quantity": 2
      }
    ],
    "shippingAddress": {
      "fullName": "Jan Kowalski",
      "line1": "Example Street 12",
      "city": "Warsaw",
      "postalCode": "00-001",
      "country": "Poland"
    },
    "totals": {
      "subtotalAmount": 49998,
      "shippingAmount": 0,
      "totalAmount": 49998,
      "currency": "PLN"
    },
    "paymentId": "66500000000000000000040",
    "createdAt": "2026-05-30T10:00:00.000Z",
    "updatedAt": "2026-05-30T10:00:00.000Z"
  }
]
```

9.10 GET /orders/:id

Access: Authenticated regular user

Description: Returns one order owned by the currently authenticated user.

Path parameters:

- id — order ObjectId.

Headers:

```
Authorization: Bearer <accessToken>
```

Request body: None.

Important rules:

- The endpoint returns only an order belonging to the authenticated user.
- If the order does not exist or belongs to another user, the response is 404 Not Found.

Example request:

```
GET /orders/6650000000000000000000000030  
Authorization: Bearer <accessToken>
```

Example success response:

```
{
  "id": "66500000000000000000030",
  "userId": "66500000000000000000001",
  "status": "pending_payment",
  "purchaseType": "private",
  "invoice": {
    "requested": false,
    "billingAddressSameAsShipping": true
  },
  "items": [
    {
      "productId": "66500000000000000000010",
      "productName": "Smart Toaster 3000",
      "productSlug": "smart-toaster-3000",
      "unitPriceAmount": 24999,
      "currency": "PLN",
      "quantity": 2,
      "imageUrl": "https://example.com/toaster.jpg"
    }
  ],
  "shippingAddress": {
    "fullName": "Jan Kowalski",
    "line1": "Example Street 12",
    "line2": "Apartment 4",
    "city": "Warsaw",
    "postalCode": "00-001",
    "country": "Poland",
    "phone": "+48123456789"
  },
  "totals": {
    "subtotalAmount": 49998,
    "shippingAmount": 0,
    "totalAmount": 49998,
    "currency": "PLN"
  },
  "paymentId": "66500000000000000000040",
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:00:00.000Z"
}
```

10 Payment Endpoints

Payment endpoints expose the mock payment flow used by the shop.

Payments are created automatically during checkout. The API does not expose a direct endpoint for creating payments manually. After checkout, the returned payment can be marked as successful or failed using the mock payment endpoints.

10.1 Payment Response Shape

Payment endpoints return the following response shape:

```
{
  "id": "665000000000000000000040",
  "orderId": "665000000000000000000030",
  "userId": "665000000000000000000001",
  "provider": "mock",
  "status": "pending",
  "amount": 49998,
  "currency": "PLN",
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:00:00.000Z"
}
```

Successful mock payments may also include:

```
{
  "mockTransactionId": "550e8400-e29b-41d4-a716-446655440000",
  "paidAt": "2026-05-30T10:05:00.000Z"
}
```

Failed mock payments may also include:

```
{
  "failureReason": "Card rejected by mock provider",
  "failedAt": "2026-05-30T10:05:00.000Z"
}
```

10.2 Payment Provider Values

The current implementation supports one payment provider:

- mock.

10.3 Payment Status Values

Payments may use the following status values:

- pending,
- paid,

- failed.

10.4 Payment Access Rules

Payment access is ownership-based.

- A regular user can access only their own payment.
- An administrator can access any payment.
- A payment can be marked as successful or failed only while it has status **pending**.

10.5 POST /payments/:id/mock-success

Access: Authenticated user or administrator

Description: Marks a pending mock payment as paid.

Path parameters:

- `id` — payment `ObjectId`.

Headers:

```
Authorization: Bearer <accessToken>
```

Request body: None.

Important rules:

- The payment must exist.
- Regular users can mark only their own payments as successful.
- Administrators can mark any payment as successful.
- The payment must have status **pending**.
- The payment status is changed to **paid**.
- A mock transaction identifier is generated.
- The related order is changed from **pending_payment** to **paid**.
- The operation is transactional with the related order update.

Example request:

```
POST /payments/6650000000000000000000040/mock-success
Authorization: Bearer <accessToken>
```

Example success response:

```
{
  "id": "665000000000000000000000000040",
  "orderId": "665000000000000000000000000030",
  "userId": "665000000000000000000000000001",
  "provider": "mock",
  "status": "paid",
  "amount": 49998,
  "currency": "PLN",
  "mockTransactionId": "550e8400-e29b-41d4-a716-446655440000",
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:05:00.000Z",
  "paidAt": "2026-05-30T10:05:00.000Z"
}
```

10.6 POST /payments/:id/mock-failure

Access: Authenticated user or administrator

Description: Marks a pending mock payment as failed.

Path parameters:

- `id` — payment `ObjectId`.

Headers:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```

Request body:

```
{
  "reason": "Card rejected by mock provider"
}
```

Body fields:

- `reason` — optional string describing why the mock payment failed.

Important rules:

- The payment must exist.
- Regular users can mark only their own payments as failed.
- Administrators can mark any payment as failed.
- The payment must have status `pending`.
- If `reason` is omitted or blank, the backend uses `Mock payment failed`.
- The payment status is changed to `failed`.
- The related order is changed from `pending_payment` to `payment_failed`.

- The operation is transactional with the related order update.

Example request:

```
POST /payments/665000000000000000000000000040/mock-failure
Authorization: Bearer <accessToken>
Content-Type: application/json
```

```
{
  "reason": "Card rejected by mock provider"
}
```

Example success response:

```
{
  "id": "665000000000000000000000000040",
  "orderId": "665000000000000000000000000030",
  "userId": "665000000000000000000000000001",
  "provider": "mock",
  "status": "failed",
  "amount": 49998,
  "currency": "PLN",
  "failureReason": "Card rejected by mock provider",
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:05:00.000Z",
  "failedAt": "2026-05-30T10:05:00.000Z"
}
```

10.7 Common Payment Errors

Payment not found:

```
{
  "message": "Payment not found",
  "error": "Not Found",
  "statusCode": 404
}
```

Payment belongs to another user:

```
{
  "message": "You cannot access this payment",
  "error": "Forbidden",
  "statusCode": 403
}
```

Payment is no longer pending:

```
{  
  "message": "Payment is not pending",  
  "error": "Conflict",  
  "statusCode": 409  
}
```

11 Review Endpoints

Review endpoints allow users to read product reviews and allow authenticated regular users to create, update, or delete their own reviews.

A user can review only a product they bought and paid for. Reviews are soft-deleted rather than hard-deleted. Product rating statistics are recalculated after review creation, update, or deletion.

Reviews store a small product snapshot field, `productName`, when they are created. This allows user review listing views to display the reviewed product name without reading the products collection.

11.1 Review Response Shape

Review endpoints return the following review shape:

```
{  
    "id": "66500000000000000000000050",  
    "productId": "66500000000000000000000010",  
    "userId": "6650000000000000000000000001",  
    "productName": "Smart Toaster 3000",  
    "rating": 5,  
    "title": "Great toaster",  
    "comment": "The QR code burn pattern is surprisingly accurate.",  
    "createdAt": "2026-05-30T10:00:00.000Z",  
    "updatedAt": "2026-05-30T10:00:00.000Z"  
}
```

Soft-deleted reviews may also contain:

```
{
  "deletedAt": "2026-05-30T10:10:00.000Z"
}
```

Public review listing and user review modification endpoints operate on active reviews.

11.2 Review Product Snapshot

Each review stores the product name at the time the review is created.

```
{  
    "productId": "665000000000000000000010",  
    "productName": "Smart Toaster 3000"  
}
```

Important rules:

- `productId` points to the reviewed product.
- `productName` is copied from the product during review creation.

- Later product name changes do not rewrite historical review data.
- GET /reviews/my can display product names without reading the products collection.

11.3 Review Status Values

Reviews may use the following status values internally:

- active,
- deleted.

The response shape does not expose the status field directly. Deleted reviews are normally excluded from public review listing.

11.4 GET /products/:productId/reviews

Access: Public

Description: Returns a paginated list of active reviews for a product.

Path parameters:

- productId — product ObjectId.

Query parameters:

- page — optional positive integer, default: 1.
- limit — optional positive integer, default: 20, maximum: 100.

Important rules:

- The product must exist and be active.
- Only active reviews are returned.
- Reviews are sorted from newest to oldest.
- Review items include productName as a product name snapshot.

Example request:

```
GET /products/66500000000000000000000010/reviews?page=1&limit=20
```

Example success response:

```
{
  "items": [
    {
      "id": "665000000000000000000050",
      "productId": "665000000000000000000010",
      "userId": "665000000000000000000001",
      "productName": "Smart Toaster 3000",
      "rating": 5,
      "title": "Great toaster",
      "comment": "The QR code burn pattern is surprisingly accurate.",
      "createdAt": "2026-05-30T10:00:00.000Z",
      "updatedAt": "2026-05-30T10:00:00.000Z"
    }
  ],
  "page": 1,
  "limit": 20,
  "total": 1,
  "totalPages": 1
}
```

11.5 GET /reviews/my

Access: Authenticated regular user

Description: Returns a paginated list of active reviews created by the currently authenticated user.

Headers:

```
Authorization: Bearer <accessToken>
```

Query parameters:

- `page` — optional positive integer, default: 1.
- `limit` — optional positive integer, default: 20, maximum: 100.

Important rules:

- Only users with the `user` role can use this endpoint.
- The endpoint returns only reviews owned by the authenticated user.
- Only active reviews are returned.
- Reviews are sorted from newest to oldest.
- The response includes `productName`, which was saved when the review was created.
- The endpoint can serve a simple user reviews view without reading product documents.

Example request:

```
GET /reviews/my?page=1&limit=20
Authorization: Bearer <accessToken>
```

Example success response:

```
{
  "items": [
    {
      "id": "665000000000000000000050",
      "productId": "665000000000000000000010",
      "userId": "665000000000000000000001",
      "productName": "Smart Toaster 3000",
      "rating": 5,
      "title": "Great toaster",
      "comment": "The QR code burn pattern is surprisingly accurate.",
      "createdAt": "2026-05-30T10:00:00.000Z",
      "updatedAt": "2026-05-30T10:00:00.000Z"
    }
  ],
  "page": 1,
  "limit": 20,
  "total": 1,
  "totalPages": 1
}
```

11.6 POST /products/:productId/reviews

Access: Authenticated regular user

Description: Creates a review for a product.

Path parameters:

- productId — product ObjectId.

Headers:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```

Request body:

```
{
  "rating": 5,
  "title": "Great toaster",
  "comment": "The QR code burn pattern is surprisingly accurate."
}
```

Body fields:

- rating — required integer from 1 to 5.

- `title` — optional non-empty string.
- `comment` — optional non-empty string.

Important rules:

- Only users with the `user` role can create reviews.
- The product must exist and be active.
- The product name is saved into the review as `productName`.
- The user must have a paid, shipped, or completed order containing the reviewed product.
- A user can have only one active review per product.
- If a previous review was deleted, creating a new active review for the same product is allowed.
- After review creation, product `averageRating` and `reviewCount` are recalculated.

Example request:

```
POST /products/6650000000000000000010/reviews
Authorization: Bearer <accessToken>
Content-Type: application/json
```

```
{
  "rating": 5,
  "title": "Great toaster",
  "comment": "The QR code burn pattern is surprisingly accurate."
}
```

Example success response:

```
{
  "id": "66500000000000000000050",
  "productId": "66500000000000000000010",
  "userId": "66500000000000000000001",
  "productName": "Smart Toaster 3000",
  "rating": 5,
  "title": "Great toaster",
  "comment": "The QR code burn pattern is surprisingly accurate.",
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:00:00.000Z"
}
```

Example failure when the product was not bought and paid for:

```
{
  "message": "You can review only products you have bought and paid for",
  "error": "Forbidden",
  "statusCode": 403
}
```

Example failure when the user already has an active review:

```
{
  "message": "You have already reviewed this product",
  "error": "Conflict",
  "statusCode": 409
}
```

11.7 PATCH /reviews/:id

Access: Authenticated regular user

Description: Updates the authenticated user's own active review.

Path parameters:

- id — review ObjectId.

Headers:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```

Request body:

```
{
  "rating": 4,
  "title": "Updated review title",
  "comment": "Updated review comment."
}
```

Body fields:

- `rating` — optional integer from 1 to 5.
- `title` — optional non-empty string.
- `comment` — optional non-empty string.

Important rules:

- At least one editable field must be provided.
- Only the owner of the review can update it.
- Only active reviews can be updated.
- `productName` is not editable through this endpoint.
- After review update, product `averageRating` and `reviewCount` are recalculated.

Example request:

```
PATCH /reviews/665000000000000000000050
Authorization: Bearer <accessToken>
Content-Type: application/json
```

```
{
  "rating": 4,
  "comment": "Updated review comment."
}
```

Example success response:

```
{
  "id": "665000000000000000000050",
  "productId": "665000000000000000000010",
  "userId": "665000000000000000000001",
  "productName": "Smart Toaster 3000",
  "rating": 4,
  "title": "Great toaster",
  "comment": "Updated review comment.",
  "createdAt": "2026-05-30T10:00:00.000Z",
  "updatedAt": "2026-05-30T10:15:00.000Z"
}
```

Example failure when editing another user's review:

```
{
  "message": "You cannot modify this review",
  "error": "Forbidden",
  "statusCode": 403
}
```

11.8 DELETE /reviews/:id

Access: Authenticated regular user

Description: Soft-deletes the authenticated user's own active review.

Path parameters:

- id — review ObjectId.

Headers:

```
Authorization: Bearer <accessToken>
```

Request body: None.

Important rules:

- Only the owner of the review can delete it.
- Only active reviews can be deleted.
- The review status is changed to `deleted`.
- The review receives a `deletedAt` timestamp.
- After review deletion, product `averageRating` and `reviewCount` are recalculated.
- Successful deletion returns 204 No Content.

Example request:

```
DELETE /reviews/66500000000000000000050
Authorization: Bearer <accessToken>
```

11.9 DELETE /reviews/admin/:id

Access: Administrator

Description: Soft-deletes any active review as an administrator.

Path parameters:

- id — review ObjectId.

Headers:

```
Authorization: Bearer <adminAccessToken>
```

Request body: None.

Important rules:

- Only administrators can use this endpoint.
- Only active reviews can be deleted.
- The review status is changed to **deleted**.
- The review receives a **deletedAt** timestamp.
- After review deletion, product **averageRating** and **reviewCount** are recalculated.
- Successful deletion returns 204 No Content.

Example request:

```
DELETE /reviews/admin/66500000000000000000050
Authorization: Bearer <adminAccessToken>
```

11.10 Common Review Errors

Product not found:

```
{
  "message": "Product not found",
  "error": "Not Found",
  "statusCode": 404
}
```

Review not found:

```
{
  "message": "Review not found",
  "error": "Not Found",
  "statusCode": 404
}
```

User did not buy and pay for the product:

```
{
  "message": "You can review only products you have bought and paid for",
  "error": "Forbidden",
  "statusCode": 403
}
```

Duplicate active review:

```
{
  "message": "You have already reviewed this product",
  "error": "Conflict",
  "statusCode": 409
}
```